ASSIGNMENT # 2

NADRA Citizen Database & CNIC Management System

A .NET Windows Forms application with an OpenCV visual-effects module

Subject	Visual Programming
Assignment	Assignment # 2 (CLO - 2)
Class	BSE – 5A
Department	Software Engineering Department
GitHub	github.com/Umer-Iftikhar/NadraAssignment2

Group Members

#	Name	Enrollment No.
1	Umer Iftikhar	09-131242-089
2	Qasim Abdul Qadeer Khan	09-131242-075

1. Introduction

This is a C# Windows Forms (.NET 8) application for a simplified NADRA citizen database. An operator logs in, registers citizens, and searches, updates or deletes their records. The data is stored, through the Dapper micro-ORM, in a SQLite database file (`nadra.db`). The file is created automatically on first run, so no database server is needed. A separate **OpenCV Image Lab** form uses the OpenCvSharp library to apply visual effects to images.
Source code: https://github.com/Umer-Iftikhar/NadraAssignment2

2. Requirements

2.1 Functional Requirements

ID	Requirement
FR-1	The operator must log in with a username and password. The app closes after 3 wrong attempts.
FR-2	The system shall register a citizen with name, father name, gender, date of birth, province, marital status, height, disability, phone, address, CNIC, issue date and photo.
FR-3	The system shall validate input: letters-only names, age 18 or above, masked phone and CNIC, required province and address.
FR-4	The CNIC must follow NADRA rules: format 12345-1234567-1, first digit = province code, last digit odd for male and even for female.
FR-5	Each CNIC must be unique. A duplicate CNIC cannot be registered, and a CNIC cannot be changed after it is issued.
FR-6	The card expiry date shall be calculated automatically: 10 years after issue, or lifetime for citizens aged 60 or above.
FR-7	The system shall search citizens by name, father name or CNIC and show the results in a grid.
FR-8	The system shall load a selected record for editing and save the changes (Update).
FR-9	The system shall delete a selected record after confirmation.
FR-10	The status bar shall show the total number of citizens and a live clock.
FR-11	The OpenCV module shall open an image and apply these effects: Grayscale, Blur, Sharpen, Edge Detection, Pencil Sketch, Cartoon, Sepia and Invert, with an adjustable brightness setting. The result can be saved.

2.2 Non-Functional Requirements

Type	Requirement
Usability	Simple tabbed interface, menu shortcuts (Ctrl+N, Ctrl+F, Ctrl+O), tooltips and error icons next to invalid fields.
Security	Parameterized SQL queries through Dapper (protection against SQL injection) and a login before access.
Data integrity	The CNIC column is UNIQUE in the database.
Portability	Embedded SQLite database; runs on any Windows PC with the .NET 8 runtime.

3. Toolbox Controls Used

Control / Component	Used for
Form, MenuStrip, StatusStrip, Timer	Main window, menus with shortcuts, record count and live clock
TabControl, GroupBox, Panel	Register / Search tabs, grouping of fields
Label, TextBox, RichTextBox	Captions, names, address
MaskedTextBox	CNIC (00000-0000000-0) and phone (0000-0000000)

Control / Component	Used for
RadioButton, CheckBox	Gender; disability, show password
ComboBox, NumericUpDown, DateTimePicker	Province and marital status, height, date of birth and issue date
PictureBox, OpenFileDialog, SaveFileDialog	Citizen photo, original and processed images
Button, MessageBox	Save, Update, Clear, Edit, Delete, confirmations
DataGridView, ContextMenuStrip	Search results with a right-click Edit/Delete menu
ListBox, TrackBar	OpenCV effect list and brightness slider
ErrorProvider, ToolTip	Validation error icons and hints

4. Project Structure

```
NadraAssignment2/
├── NadraSystem.sln
└── NadraSystem/
    ├── NadraSystem.csproj      NuGet: Dapper, Microsoft.Data.Sqlite, OpenCvSharp4 (+ runtime.win, Extensic
    ├── Program.cs              Entry point: login, then main form
    ├── Citizen.cs              Citizen model class
    ├── Database.cs             Dapper + SQLite: create table, sample data, CRUD, search
    └── Forms/
        ├── LoginForm.cs (+ .Designer.cs)     Login screen
        ├── MainForm.cs  (+ .Designer.cs)     Register and Search tabs
        └── OpenCvForm.cs (+ .Designer.cs)    OpenCV visual effects
```

How to run: open `NadraSystem.sln` in Visual Studio and press F5. The NuGet packages are restored automatically. Login: `admin / admin123`.

5. Source Code

Only the hand-written code is listed. The `*.Designer.cs` files hold the control layout generated by the Visual Studio designer and are included in the project folder.

Program.cs

```
using NadraSystem.Forms;

namespace NadraSystem;

internal static class Program
{
    [STAThread]
    static void Main()
    {
        ApplicationConfiguration.Initialize();
        Database.Initialize();                      // creates nadra.db on first run

        using var login = new LoginForm();
        if (login.ShowDialog() == DialogResult.OK)
            Application.Run(new MainForm());
    }
}
```

Citizen.cs

```
namespace NadraSystem;

/// <summary>One citizen record in the NADRA database.</summary>
public class Citizen
{
    public int Id { get; set; }
    public string Cnic { get; set; }
    public string Name { get; set; }
    public string FatherName { get; set; }
    public string Gender { get; set; }
    public DateTime DateOfBirth { get; set; }
    public string Province { get; set; }
    public string MaritalStatus { get; set; }
    public string Phone { get; set; }
    public string Address { get; set; }
    public int HeightCm { get; set; }
    public bool IsDisabled { get; set; }
    public DateTime IssueDate { get; set; }
    public DateTime ExpiryDate { get; set; }
    public byte[] Photo { get; set; }
}
```

Database.cs

```
using Dapper;
using Microsoft.Data.Sqlite;

namespace NadraSystem;

/// <summary>Data access for the NADRA citizen database (SQLite file: nadra.db) using Dapper.</summary>
public static class Database
{
    private static readonly string ConnStr =
        "Data Source=" + Path.Combine(AppContext.BaseDirectory, "nadra.db");

    private static SqliteConnection Open() => new SqliteConnection(ConnStr);   // Dapper opens/closes it

    public static void Initialize()
    {
        using var con = Open();
        con.Execute(@"CREATE TABLE IF NOT EXISTS Citizens(
            Id INTEGER PRIMARY KEY AUTOINCREMENT,
            Cnic TEXT NOT NULL UNIQUE, Name TEXT NOT NULL, FatherName TEXT NOT NULL,
            Gender TEXT, DateOfBirth TEXT, Province TEXT, MaritalStatus TEXT, Phone TEXT,
            Address TEXT, HeightCm INTEGER, IsDisabled INTEGER,
            IssueDate TEXT, ExpiryDate TEXT, Photo BLOB)");

        // sample records on first run
        if (con.ExecuteScalar<int>("SELECT COUNT(*) FROM Citizens") == 0)
        {
            Seed("35202-1234567-1", "Ahmed Raza", "Raza Hussain", "Male", 1991, "Punjab", "Married");
            Seed("61101-7654321-2", "Fatima Noor", "Noor Muhammad", "Female", 1997, "Islamabad", "Single");
            Seed("17301-2345678-3", "Bilal Khan", "Sher Khan", "Male", 1985, "Khyber Pakhtunkhwa", "Married");
            Seed("42101-3456789-4", "Sana Memon", "Ibrahim Memon", "Female", 2001, "Sindh", "Single");
```

```
            Seed("54400-4567890-5", "Usman Baloch", "Akbar Baloch", "Male", 1979, "Balochistan", "Widowed");
        }
    }

    private static void Seed(string cnic, string name, string father, string gender, int year, string prov, string marital)
    {
        var issue = new DateTime(2022, 3, 15);
        Insert(new Citizen
        {
            Cnic = cnic, Name = name, FatherName = father, Gender = gender,
            DateOfBirth = new DateTime(year, 6, 1), Province = prov, MaritalStatus = marital,
            Phone = "0300-1234567", Address = "House 1, Street 1, " + prov, HeightCm = 170,
            IssueDate = issue, ExpiryDate = issue.AddYears(10)
        });
    }

    // Dapper maps the @Parameters to the Citizen properties with the same names
    public static void Insert(Citizen c)
    {
        using var con = Open();
        con.Execute(@"INSERT INTO Citizens(Cnic, Name, FatherName, Gender, DateOfBirth, Province, MaritalStatus,
                                           Phone, Address, HeightCm, IsDisabled, IssueDate, ExpiryDate, Photo)
                      VALUES(@Cnic, @Name, @FatherName, @Gender, @DateOfBirth, @Province, @MaritalStatus,
                             @Phone, @Address, @HeightCm, @IsDisabled, @IssueDate, @ExpiryDate, @Photo)", c);
    }

    public static void Update(Citizen c)
    {
        using var con = Open();
        con.Execute(@"UPDATE Citizens SET Name=@Name, FatherName=@FatherName, Gender=@Gender,
                      DateOfBirth=@DateOfBirth, Province=@Province, MaritalStatus=@MaritalStatus, Phone=@Phone,
                      Address=@Address, HeightCm=@HeightCm, IsDisabled=@IsDisabled, IssueDate=@IssueDate,
                      ExpiryDate=@ExpiryDate, Photo=@Photo
                      WHERE Cnic=@Cnic", c);
    }

    public static void Delete(string cnic)
    {
        using var con = Open();
        con.Execute("DELETE FROM Citizens WHERE Cnic=@cnic", new { cnic });
    }

    public static bool Exists(string cnic)
    {
        using var con = Open();
        return con.ExecuteScalar<int>("SELECT COUNT(*) FROM Citizens WHERE Cnic=@cnic", new { cnic }) > 0;
    }

    public static Citizen GetByCnic(string cnic)
    {
        using var con = Open();
        return con.QueryFirstOrDefault<Citizen>("SELECT * FROM Citizens WHERE Cnic=@cnic", new { cnic });
    }

    /// <summary>Searches by name, father name or CNIC.</summary>
    public static List<Citizen> Search(string text)
    {
        using var con = Open();
        return con.Query<Citizen>(@"SELECT * FROM Citizens
                                    WHERE Name LIKE @t OR FatherName LIKE @t OR Cnic LIKE @t
                                    ORDER BY Name", new { t = "%" + text + "%" }).ToList();
    }
}
```

Forms/LoginForm.cs

```
namespace NadraSystem.Forms;

public partial class LoginForm : Form
{
    private int attempts = 0;

    public LoginForm()
    {
        InitializeComponent();
    }

    private void chkShow_CheckedChanged(object sender, EventArgs e)
    {
        txtPassword.UseSystemPasswordChar = !chkShow.Checked;
    }

    private void btnLogin_Click(object sender, EventArgs e)
    {
```

```
        if (txtUsername.Text == "admin" && txtPassword.Text == "admin123")
        {
            DialogResult = DialogResult.OK;
            return;
        }

        attempts++;
        lblError.Text = $"Invalid username or password ({3 - attempts} attempts left)";
        if (attempts == 3)
        {
            MessageBox.Show("Too many failed attempts. The application will close.", "Login",
                MessageBoxButtons.OK, MessageBoxIcon.Stop);
            DialogResult = DialogResult.Cancel;
        }
    }

    private void btnExit_Click(object sender, EventArgs e) => DialogResult = DialogResult.Cancel;
}
```

Forms/MainForm.cs

```
using System.Drawing.Imaging;
using System.Text.RegularExpressions;

namespace NadraSystem.Forms;

public partial class MainForm : Form
{
    // first digit of a CNIC tells the province
    private readonly Dictionary<string, char> provinceCodes = new()
    {
        ["Khyber Pakhtunkhwa"] = '1', ["Punjab"] = '3', ["Sindh"] = '4', ["Balochistan"] = '5',
        ["Islamabad"] = '6', ["Gilgit-Baltistan"] = '7', ["Azad Kashmir"] = '8'
    };
    private byte[] photo;

    public MainForm()
    {
        InitializeComponent();
    }

    private void MainForm_Load(object sender, EventArgs e)
    {
        cmbProvince.Items.AddRange(provinceCodes.Keys.ToArray());
        cmbMarital.Items.AddRange(new object[] { "Single", "Married", "Divorced", "Widowed" });
        cmbMarital.SelectedIndex = 0;
        dtpDob.Value = DateTime.Today.AddYears(-20);
        dtpDob.MaxDate = DateTime.Today;
        toolTip1.SetToolTip(mtxtCnic, "First digit = province code, last digit odd for male / even for female");
        toolTip1.SetToolTip(btnBrowse, "Choose the citizen's photograph");
        timer1.Start();
        LoadGrid();
    }

    // ---------------- helpers
    private int Age
    {
        get
        {
            int age = DateTime.Today.Year - dtpDob.Value.Year;
            if (dtpDob.Value.Date > DateTime.Today.AddYears(-age)) age--;
            return age;
        }
    }

    private DateTime Expiry => Age >= 60 ? DateTime.MaxValue.Date : dtpIssue.Value.Date.AddYears(10);

    private void LoadGrid()
    {
        var list = Database.Search(txtSearch.Text.Trim());
        dgvCitizens.DataSource = list.Select(c => new
        {
            c.Cnic, c.Name, Father = c.FatherName, c.Gender, DOB = c.DateOfBirth.ToString("dd-MM-yyyy"),
            c.Province, c.Phone, Expiry = c.ExpiryDate == DateTime.MaxValue.Date ? "Lifetime" : c.ExpiryDate.ToString("dd-MM-
    yyyy")
        }).ToList();
        lblCount.Text = $"{list.Count} record(s) found";
        lblRecords.Text = $"Total citizens: {Database.Search("").Count}";
    }

    // ---------------- events
    private void timer1_Tick(object sender, EventArgs e) => lblClock.Text = DateTime.Now.ToString("dd-MMM-yyyy  hh:mm:ss
    tt");
    private void dtpDob_ValueChanged(object sender, EventArgs e)
```

```
{
    lblAge.Text = $"Age: {Age} years";
    lblAge.ForeColor = Age < 18 ? Color.Red : Color.Green;
    dtpIssue_ValueChanged(sender, e);
}

private void dtpIssue_ValueChanged(object sender, EventArgs e)
{
    lblExpiry.Text = Age >= 60 ? "Expiry: LIFETIME" : $"Expiry: {Expiry:dd-MM-yyyy}";
}

private void btnBrowse_Click(object sender, EventArgs e)
{
    if (openFileDialog1.ShowDialog() != DialogResult.OK) return;
    using var img = Image.FromFile(openFileDialog1.FileName);
    using var resized = new Bitmap(img, 240, 300);
    using var ms = new MemoryStream();
    resized.Save(ms, ImageFormat.Jpeg);
    photo = ms.ToArray();
    picPhoto.Image = Image.FromStream(new MemoryStream(photo));
}

// ---------------- validation
private bool ValidateForm()
{
    errorProvider1.Clear();
    bool ok = true;
    void Error(Control c, string msg) { errorProvider1.SetError(c, msg); ok = false; }

    if (!Regex.IsMatch(txtName.Text.Trim(), @"^[A-Za-z ]{3,50}$")) Error(txtName, "Name must contain 3-50 letters only");
    if (!Regex.IsMatch(txtFather.Text.Trim(), @"^[A-Za-z ]{3,50}$")) Error(txtFather, "Father name must contain 3-50
letters only");
    if (!rbMale.Checked && !rbFemale.Checked) Error(grpGender, "Select gender");
    if (Age < 18) Error(dtpDob, "Citizen must be at least 18 years old");
    if (cmbProvince.SelectedIndex < 0) Error(cmbProvince, "Select province");
    if (!mtxtPhone.MaskCompleted) Error(mtxtPhone, "Enter phone as 03XX-XXXXXXX");
    if (rtbAddress.Text.Trim().Length < 10) Error(rtbAddress, "Enter complete address");

    if (!mtxtCnic.MaskCompleted) Error(mtxtCnic, "Enter CNIC as 12345-1234567-1");
    else
    {
        bool oddLastDigit = (mtxtCnic.Text[^1] - '0') % 2 == 1;
        if (rbMale.Checked && !oddLastDigit) Error(mtxtCnic, "Male CNIC must end with an odd digit");
        if (rbFemale.Checked && oddLastDigit) Error(mtxtCnic, "Female CNIC must end with an even digit");
        if (cmbProvince.SelectedIndex >= 0 && mtxtCnic.Text[0] != provinceCodes[cmbProvince.Text])
            Error(mtxtCnic, $"CNIC for {cmbProvince.Text} must start with {provinceCodes[cmbProvince.Text]}");
    }
    return ok;
}

private Citizen ReadForm() => new Citizen
{
    Cnic = mtxtCnic.Text,
    Name = txtName.Text.Trim(),
    FatherName = txtFather.Text.Trim(),
    Gender = rbMale.Checked ? "Male" : "Female",
    DateOfBirth = dtpDob.Value.Date,
    Province = cmbProvince.Text,
    MaritalStatus = cmbMarital.Text,
    Phone = mtxtPhone.Text,
    Address = rtbAddress.Text.Trim(),
    HeightCm = (int)nudHeight.Value,
    IsDisabled = chkDisabled.Checked,
    IssueDate = dtpIssue.Value.Date,
    ExpiryDate = Expiry,
    Photo = photo
};

// ---------------- CRUD buttons
private void btnSave_Click(object sender, EventArgs e)
{
    if (!ValidateForm()) return;
    if (Database.Exists(mtxtCnic.Text))
    {
        errorProvider1.SetError(mtxtCnic, "This CNIC is already registered");
        return;
    }
    Database.Insert(ReadForm());
    MessageBox.Show("Citizen registered successfully.\nCNIC: " + mtxtCnic.Text, "Saved",
        MessageBoxButtons.OK, MessageBoxIcon.Information);
    ClearForm();
    LoadGrid();
}

private void btnUpdate_Click(object sender, EventArgs e)
```

```
    {
        if (!ValidateForm()) return;
        if (!Database.Exists(mtxtCnic.Text))
        {
            MessageBox.Show("No record with this CNIC. Use Save to register.");
            return;
        }
        Database.Update(ReadForm());
        MessageBox.Show("Record updated successfully.", "Updated", MessageBoxButtons.OK, MessageBoxIcon.Information);
        ClearForm();
        LoadGrid();
    }

    private void btnClear_Click(object sender, EventArgs e) => ClearForm();

    private void ClearForm()
    {
        txtName.Clear(); txtFather.Clear(); rtbAddress.Clear(); mtxtPhone.Clear();
        mtxtCnic.Clear(); mtxtCnic.ReadOnly = false;
        rbMale.Checked = rbFemale.Checked = chkDisabled.Checked = false;
        cmbProvince.SelectedIndex = -1;
        nudHeight.Value = 170;
        dtpDob.Value = DateTime.Today.AddYears(-20);
        photo = null; picPhoto.Image = null;
        errorProvider1.Clear();
    }

    private void btnSearch_Click(object sender, EventArgs e) => LoadGrid();

    private string SelectedCnic => dgvCitizens.CurrentRow?.Cells["Cnic"].Value?.ToString();

    // loads the selected record into the Register tab for editing
    private void btnEdit_Click(object sender, EventArgs e)
    {
        var c = Database.GetByCnic(SelectedCnic);
        if (c == null) return;
        ClearForm();
        mtxtCnic.Text = c.Cnic; mtxtCnic.ReadOnly = true;     // CNIC can't be changed
        txtName.Text = c.Name; txtFather.Text = c.FatherName;
        rbMale.Checked = c.Gender == "Male"; rbFemale.Checked = c.Gender == "Female";
        dtpDob.Value = c.DateOfBirth; cmbProvince.Text = c.Province; cmbMarital.Text = c.MaritalStatus;
        mtxtPhone.Text = c.Phone; rtbAddress.Text = c.Address; nudHeight.Value = c.HeightCm;
        chkDisabled.Checked = c.IsDisabled; dtpIssue.Value = c.IssueDate;
        photo = c.Photo;
        picPhoto.Image = photo == null ? null : Image.FromStream(new MemoryStream(photo));
        tabControl1.SelectedTab = tabRegister;
    }

    private void btnDelete_Click(object sender, EventArgs e)
    {
        if (SelectedCnic == null) return;
        if (MessageBox.Show($"Delete citizen {SelectedCnic}?", "Confirm Delete",
                MessageBoxButtons.YesNo, MessageBoxIcon.Warning) == DialogResult.Yes)
        {
            Database.Delete(SelectedCnic);
            LoadGrid();
        }
    }

    // ---------------- menu
    private void mnuRegister_Click(object sender, EventArgs e) => tabControl1.SelectedTab = tabRegister;
    private void mnuSearch_Click(object sender, EventArgs e) => tabControl1.SelectedTab = tabSearch;
    private void mnuOpenCv_Click(object sender, EventArgs e) => new OpenCvForm().ShowDialog();
    private void mnuExit_Click(object sender, EventArgs e) => Close();
    private void mnuAbout_Click(object sender, EventArgs e) =>
        MessageBox.Show("NADRA Citizen Database System\nVisual Programming - Assignment 2", "About");
}
```

Forms/OpenCvForm.cs

```
using OpenCvSharp;
using OpenCvSharp.Extensions;

namespace NadraSystem.Forms;

/// <summary>OpenCV module: applies visual effects to an image (e.g. a citizen photo).</summary>
public partial class OpenCvForm : Form
{
    private Mat original;

    public OpenCvForm()
    {
        InitializeComponent();
        lstEffects.Items.AddRange(new object[]
```

```
            { "Original", "Grayscale", "Blur", "Sharpen", "Edge Detection", "Pencil Sketch", "Cartoon", "Sepia", "Invert" });
        lstEffects.SelectedIndex = 0;
    }

    private void btnOpen_Click(object sender, EventArgs e)
    {
        if (openFileDialog1.ShowDialog() != DialogResult.OK) return;
        original = Cv2.ImRead(openFileDialog1.FileName, ImreadModes.Color);   // BGR image
        picOriginal.Image = original.ToBitmap();
        ApplyEffect();
    }

    private void lstEffects_SelectedIndexChanged(object sender, EventArgs e) => ApplyEffect();

    private void trackBrightness_Scroll(object sender, EventArgs e)
    {
        lblBrightness.Text = "Brightness: " + trackBrightness.Value;
        ApplyEffect();
    }

    private void ApplyEffect()
    {
        if (original == null) return;
        Mat result = new Mat();
        Mat gray = original.CvtColor(ColorConversionCodes.BGR2GRAY);

        switch (lstEffects.Text)
        {
            case "Grayscale":
                result = gray.CvtColor(ColorConversionCodes.GRAY2BGR);
                break;
            case "Blur":
                Cv2.GaussianBlur(original, result, new OpenCvSharp.Size(21, 21), 0);
                break;
            case "Sharpen":
                var kernel = Mat.FromPixelData(3, 3, MatType.CV_32F, new float[] { 0, -1, 0, -1, 5, -1, 0, -1, 0 });
                Cv2.Filter2D(original, result, -1, kernel);
                break;
            case "Edge Detection":
                Cv2.Canny(gray, gray, 80, 160);
                result = gray.CvtColor(ColorConversionCodes.GRAY2BGR);
                break;
            case "Pencil Sketch":
                Cv2.PencilSketch(original, gray, result, 60, 0.07f, 0.05f);
                result = gray.CvtColor(ColorConversionCodes.GRAY2BGR);
                break;
            case "Cartoon":
                Cv2.Stylization(original, result, 60, 0.45f);
                break;
            case "Sepia":
                var sepia = Mat.FromPixelData(3, 3, MatType.CV_32F, new float[]
                    { 0.131f, 0.534f, 0.272f, 0.168f, 0.686f, 0.349f, 0.189f, 0.769f, 0.393f });
                Cv2.Transform(original, result, sepia);
                break;
            case "Invert":
                Cv2.BitwiseNot(original, result);
                break;
            default:
                result = original.Clone();
                break;
        }

        // brightness from the TrackBar (-100 .. +100)
        result.ConvertTo(result, -1, 1.0, trackBrightness.Value);
        picResult.Image = result.ToBitmap();
        lblStatus.Text = $"Effect: {lstEffects.Text}   |   Size: {original.Width} x {original.Height}";
    }

    private void btnSave_Click(object sender, EventArgs e)
    {
        if (picResult.Image != null && saveFileDialog1.ShowDialog() == DialogResult.OK)
            picResult.Image.Save(saveFileDialog1.FileName);
    }
}
```

6. Output Screenshots

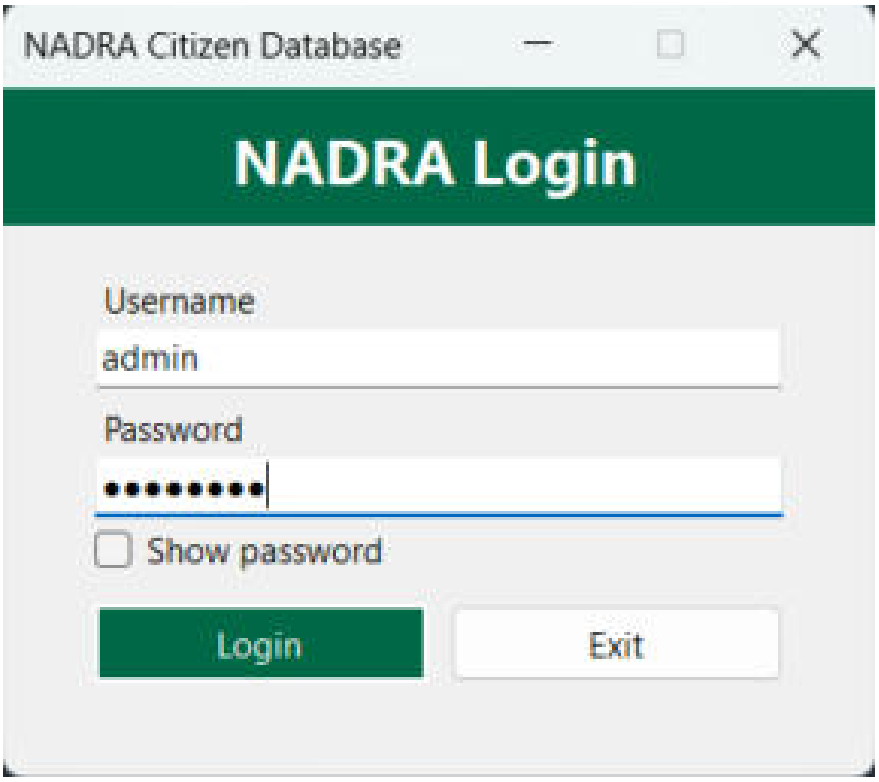


Figure 1. Login screen.

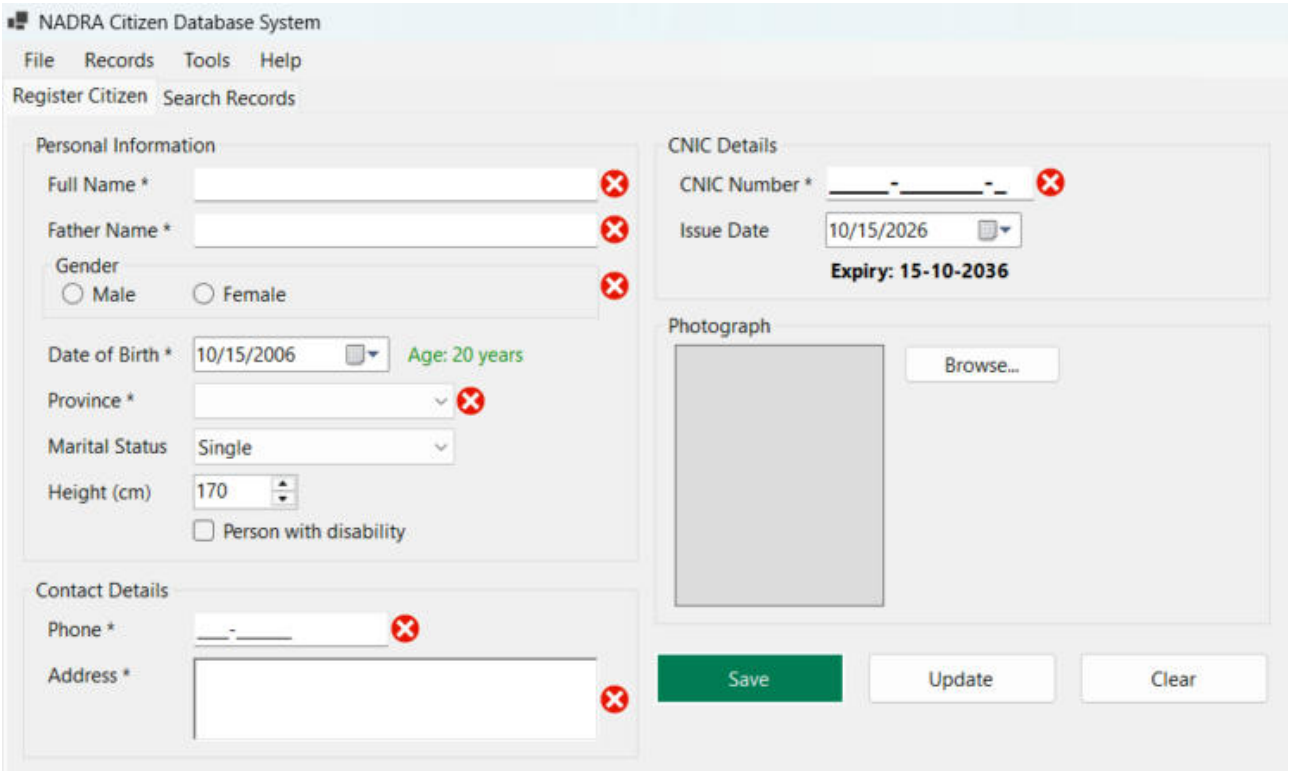


Figure 2. Validation: saving an empty form shows ErrorProvider icons on the required fields.

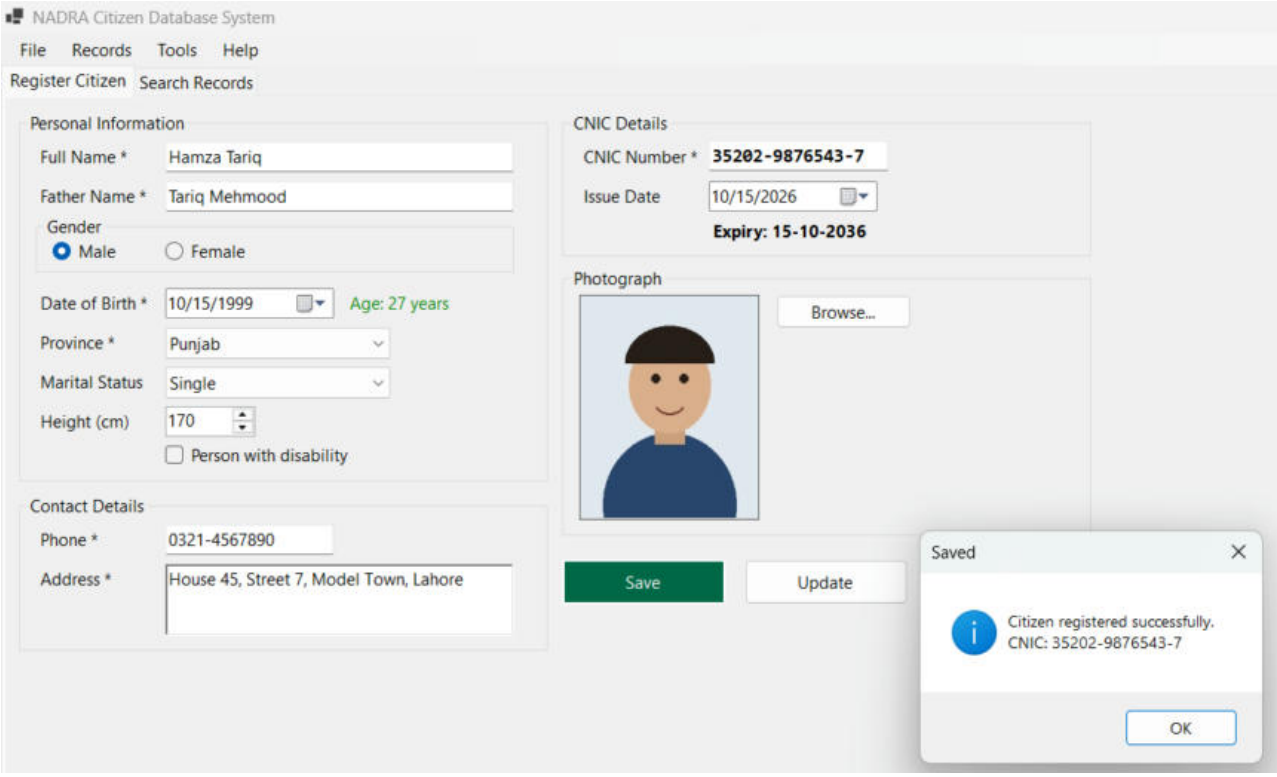


Figure 3. A new citizen registered: the CNIC follows the rules (3 = Punjab, odd last digit = male), age and expiry are calculated, and a photo is attached.

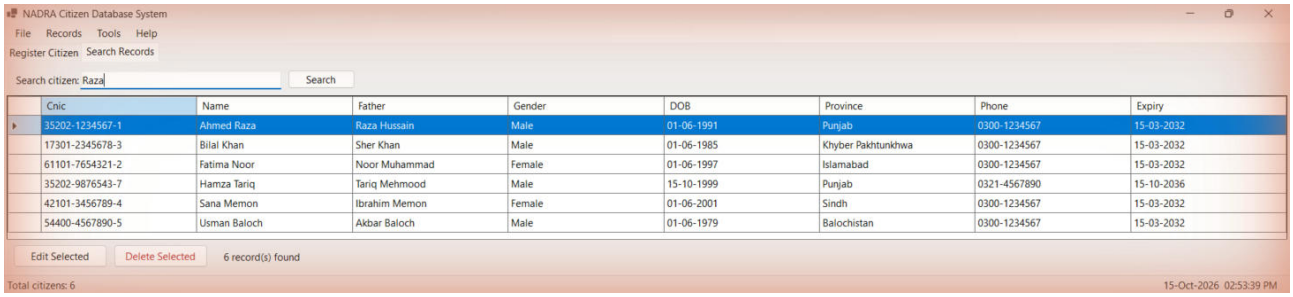


Figure 4. Search Records tab showing all citizens in the DataGridView, including the new record.



Figure 5. Searching for 'Raza' returns the matching record.

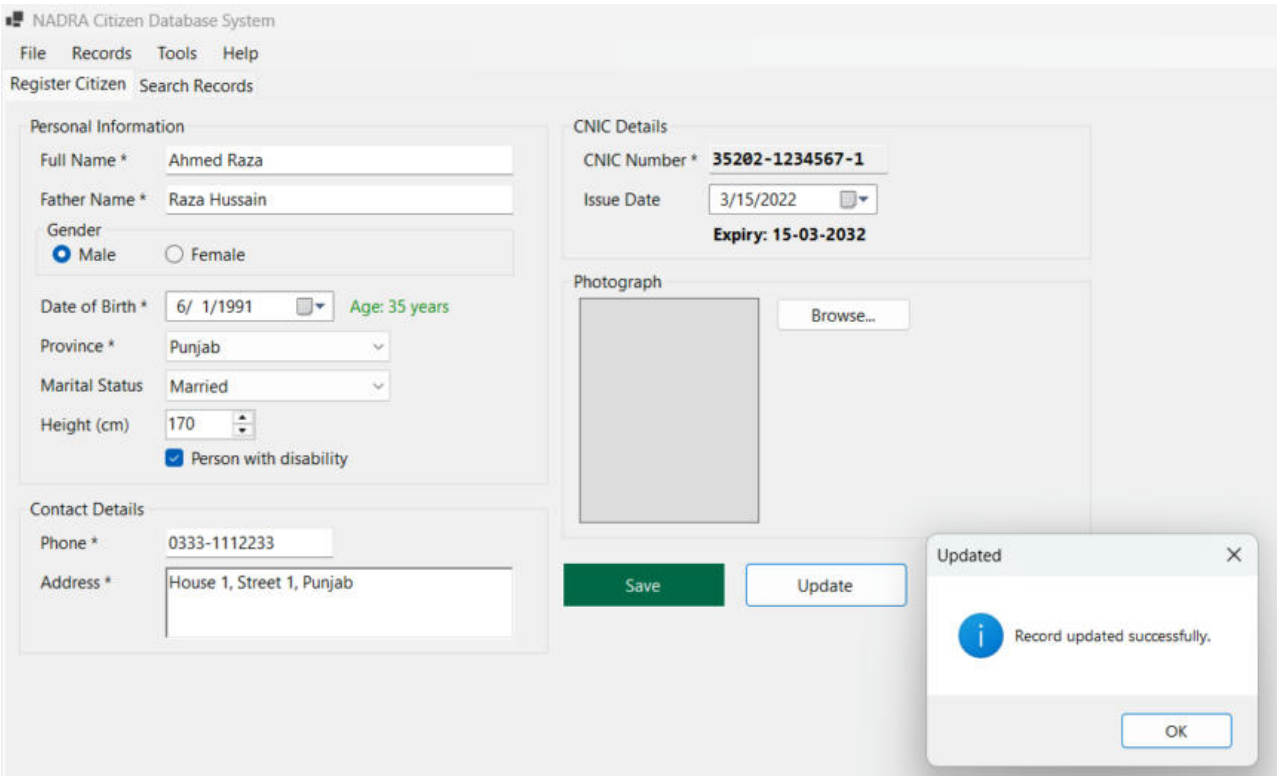


Figure 6. Edit: the record is loaded into the form (the CNIC is read-only), the phone number is changed and the record is updated.

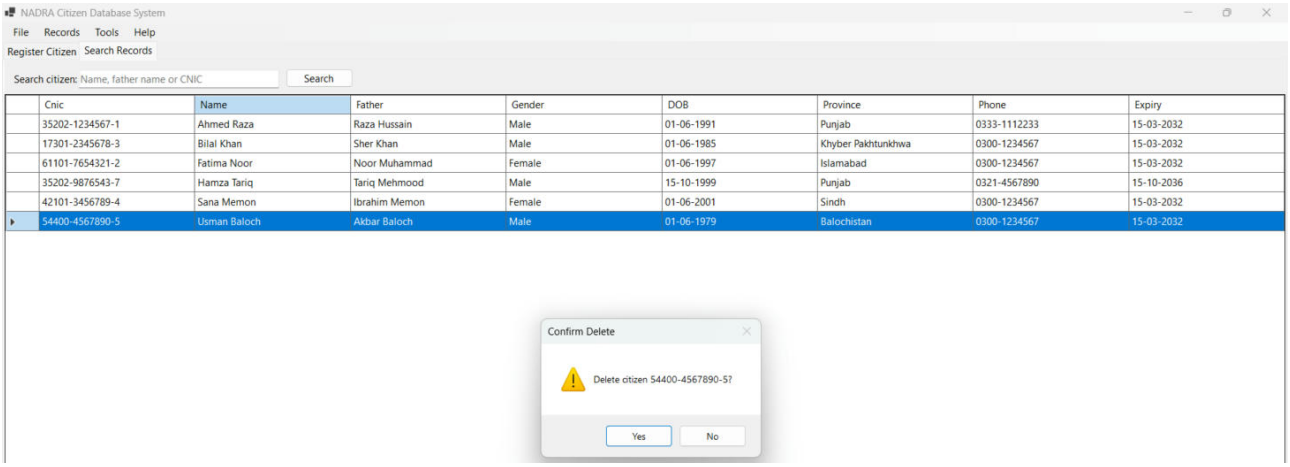


Figure 7. Delete with confirmation.

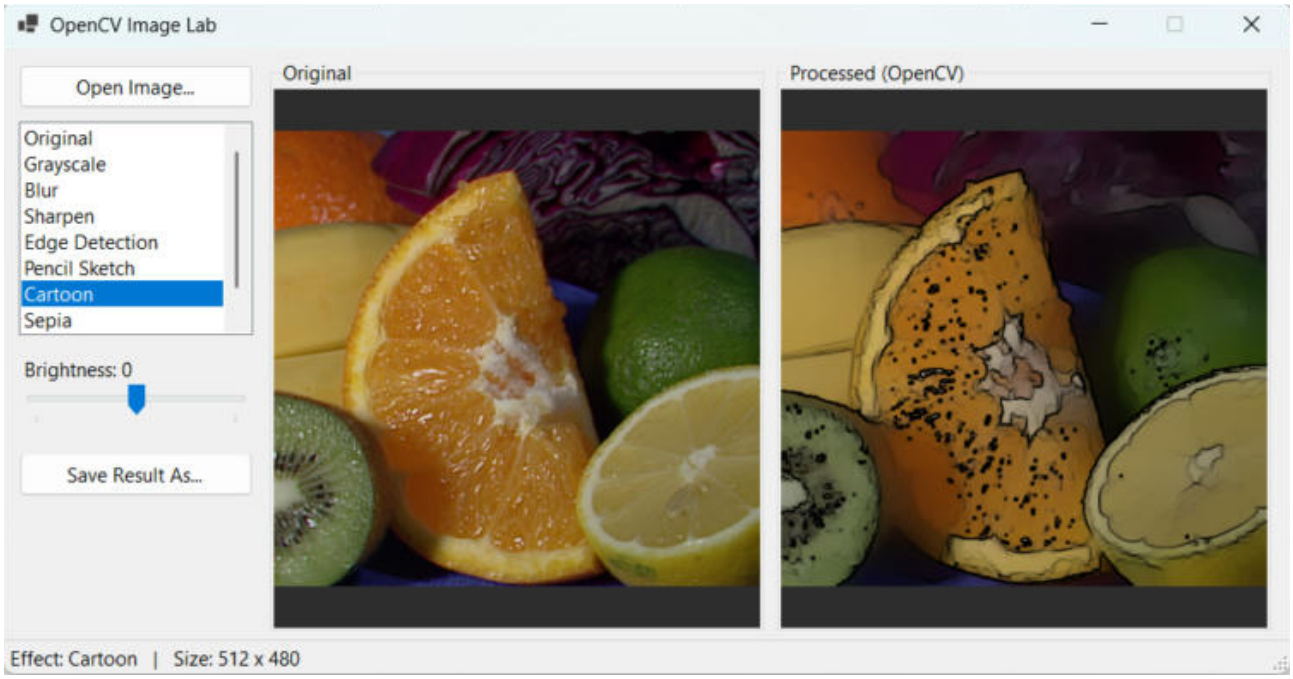


Figure 8. OpenCV Image Lab: Cartoon effect (Cv2.Stylization).



Figure 9. OpenCV: Edge Detection (Cv2.Canny).



Figure 10. OpenCV: Pencil Sketch (Cv2.PencilSketch).

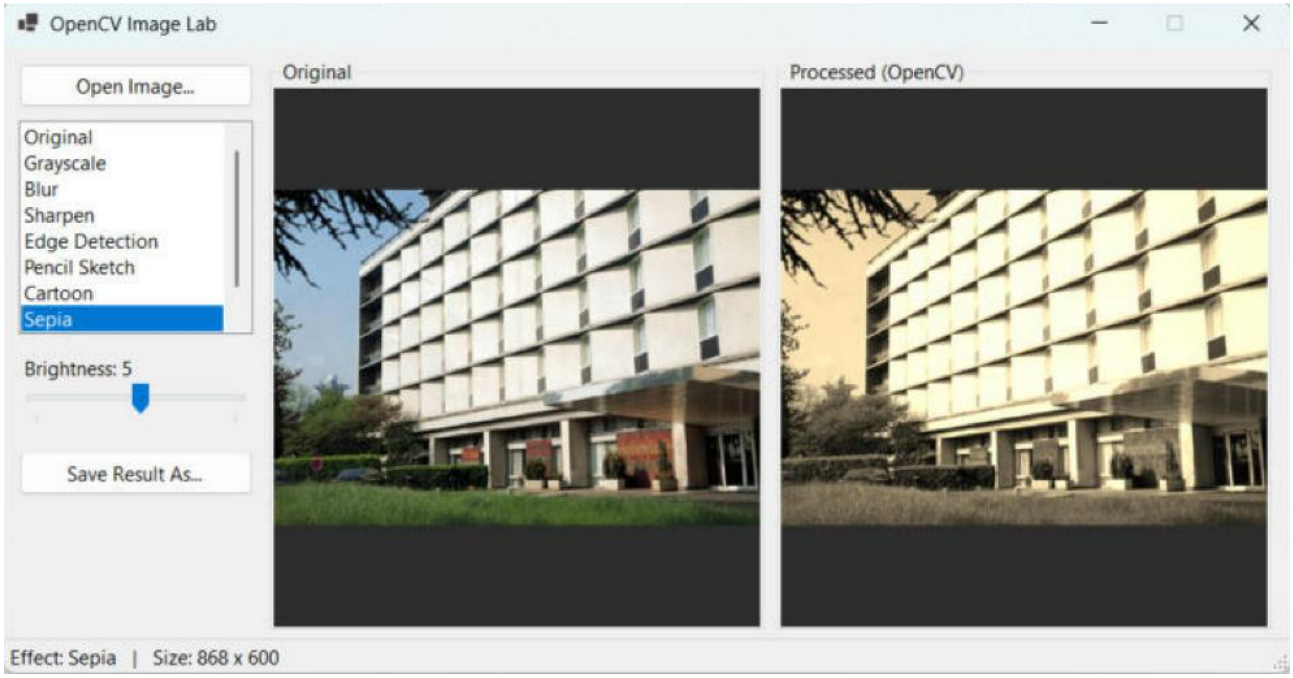


Figure 11. OpenCV: Sepia (Cv2.Transform) with the brightness TrackBar.